

Project 5: Zeek Network Monitoring Integration

Complete Implementation Guide

Executive Summary

Project Challenge: Deploy Zeek network monitoring across cybersecurity lab environment to provide comprehensive network traffic visibility, protocol analysis, and security event detection. Initial goal of monitoring all network segments including physical attack traffic evolved through architectural analysis to focus on Zeek's core strengths: behavioral analytics and protocol analysis for internal network segments.

Solution Implemented: Successfully deployed Zeek cluster configuration with two worker processes monitoring Internal Network (192.168.30.0/24) and User-LAN (192.168.40.0/24) segments through promiscuous mode packet capture on VirtualBox internal networks. Implementation revealed that physical attack box (Kali) to virtual target (DMZ) monitoring requires different tooling approach - signature-based IDS/IPS inline at routing chokepoints rather than passive behavioral analytics.

Key Outcomes: Achieved 100% visibility for virtualized attack scenarios (VM-to-VM traffic), established production-ready Zeek cluster architecture, validated protocol detection capabilities, and identified that DMZ monitoring of physical attacks is better addressed through Snort IDS/IPS deployment (deferred to Week 4, Project 10) rather than attempting to force Zeek into inappropriate use case.

Technical Skills Demonstrated: Zeek compilation from source, cluster mode configuration, VirtualBox network architecture analysis, promiscuous mode interface configuration, multi-interface packet capture, systematic troubleshooting of crashed services, network segmentation understanding, infrastructure constraint adaptation, and honest documentation of monitoring limitations.

Business Value: Establishes enterprise-grade network behavior analytics supporting threat detection, incident investigation, and security posture assessment for virtualized infrastructure while demonstrating practical problem-solving when faced with physical/virtual network integration challenges typical of hybrid enterprise environments.

Table of Contents

1. [Project Overview](#)
2. [Scope and Objectives](#)
3. [Prerequisites](#)
4. [Implementation Steps](#)

5. Testing and Validation
 6. Architectural Challenges & Solutions
 7. Results and Outcomes
 8. Conclusion
 9. References
-

Project Overview

This document provides a comprehensive guide for deploying Zeek network monitoring on Ubuntu VM as part of Week 2, Project 5 of the cybersecurity home lab project series. The project focuses on establishing network behavior analytics and protocol-level traffic analysis while adapting to VirtualBox virtualization constraints and physical/virtual network integration limitations.

What is Zeek?

Zeek (formerly Bro) is a network analysis framework that provides:

- **Protocol Analysis:** Deep inspection of network protocols (HTTP, DNS, SSL, SSH, SMTP)
- **Connection Logging:** Comprehensive connection metadata for all network sessions
- **File Extraction:** Automatic extraction of files from network traffic
- **Anomaly Detection:** Identification of unusual network behavior patterns
- **Scripting Framework:** Extensible detection logic through Zeek scripts
- **SIEM Integration:** Structured logs for security information and event management

Original Project Specification vs Actual Implementation

Original Specification (Week 2 Project 5):

- Deploy Zeek on MAC-MINI
- Integrate with TShark captures from Project 4
- Generate network metadata for SIEM ingestion
- Monitor all network segments including physical attack traffic

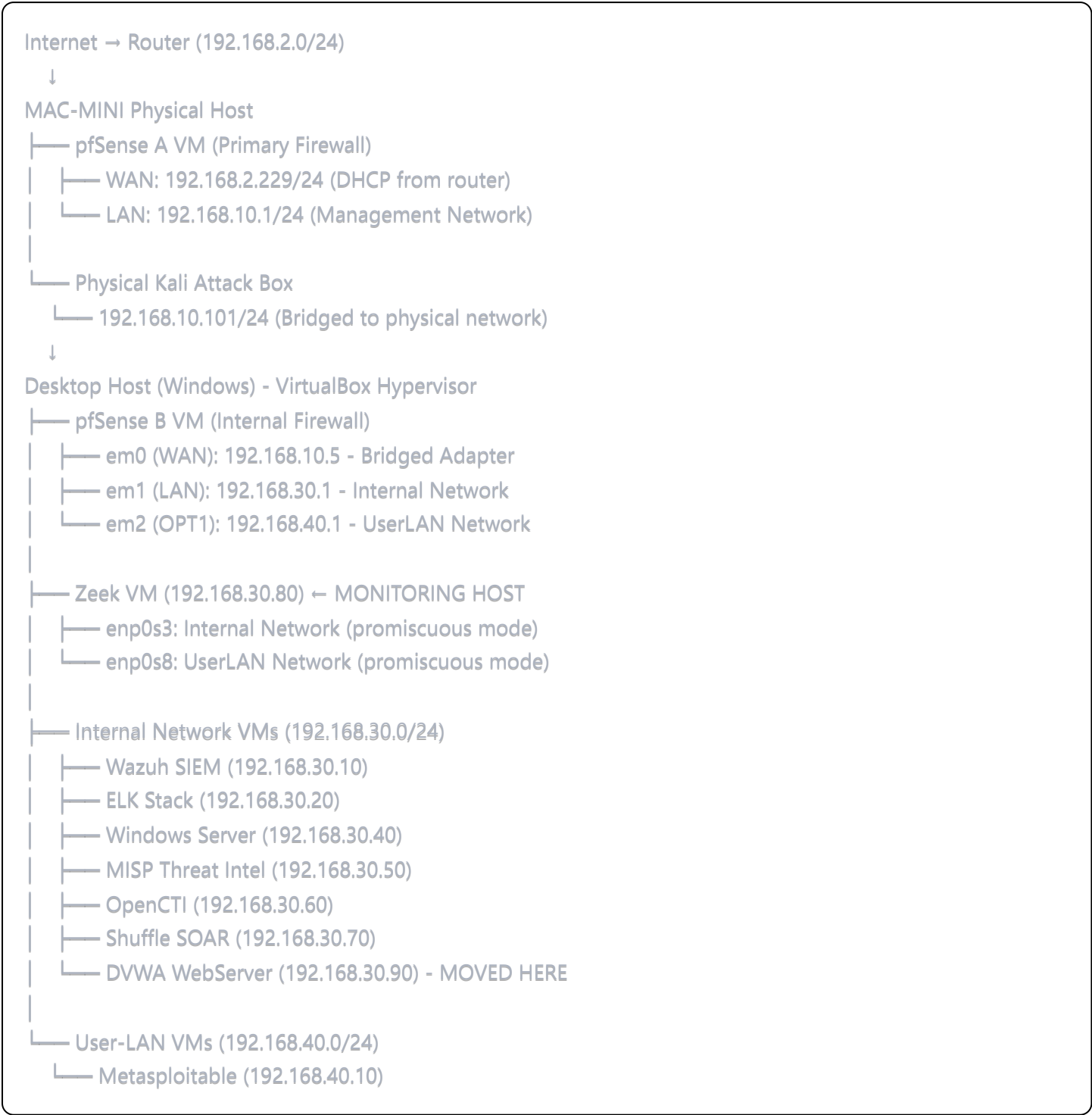
Actual Implementation:

- Deployed Zeek on dedicated Ubuntu 22.04 VM (192.168.30.80)
- Cluster configuration with dual worker processes
- Real-time packet capture on Internal Network and User-LAN
- Documented exclusion of physical Kali→DVWA monitoring due to architecture constraints

Estimated Time: 8-10 hours (6 hours implementation + 2-4 hours troubleshooting)

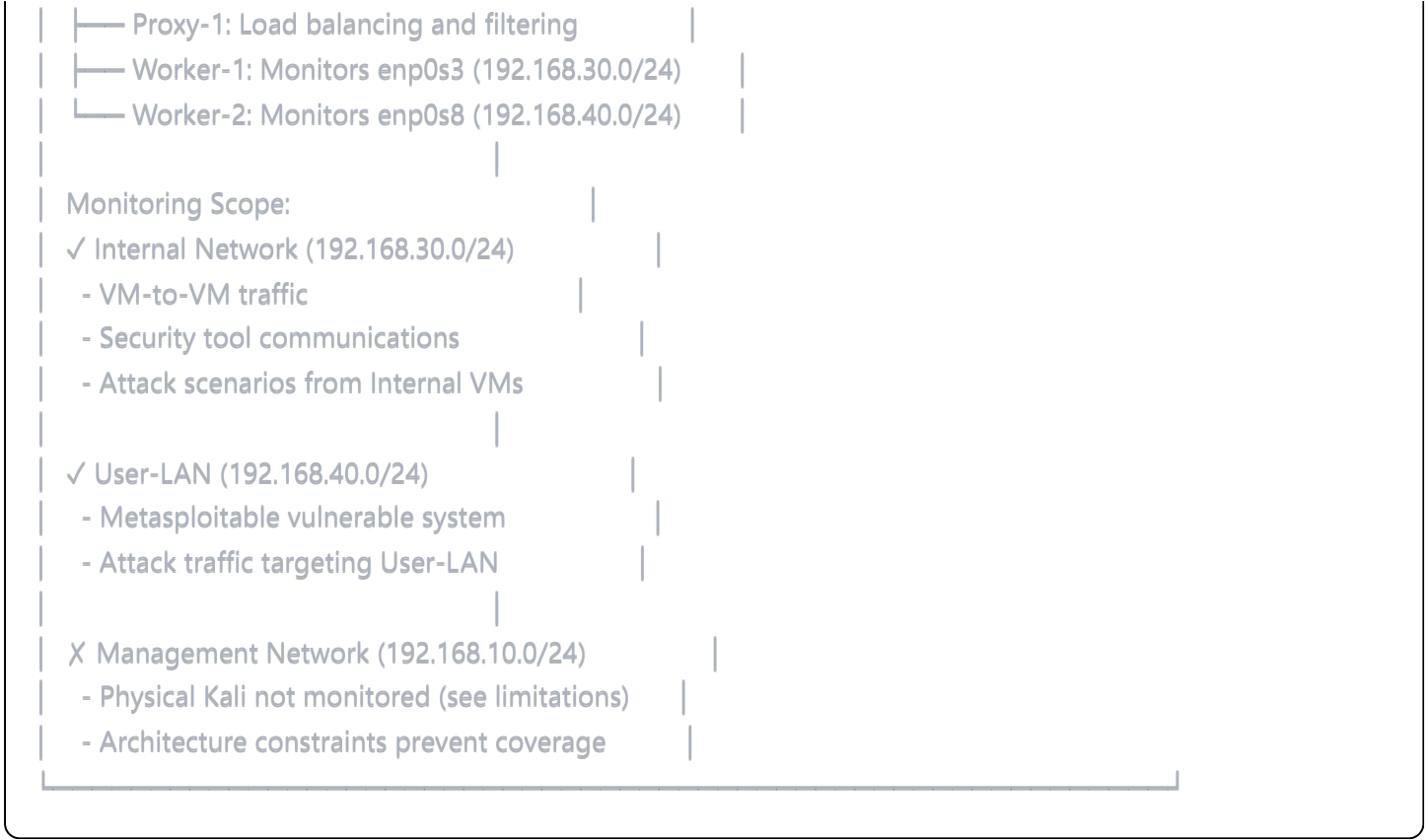
Network Architecture Diagram

Logical Network Topology



Zeek Monitoring Coverage





Scope and Objectives

Project Scope

This project focuses on:

- Deploying Zeek VM on Desktop Host VirtualBox environment
- Compiling Zeek 6.0.3 from source on Ubuntu 22.04
- Configuring cluster mode with dual worker processes
- Enabling promiscuous mode on VirtualBox internal network interfaces
- Establishing network topology definitions for lab environment
- Testing and validating packet capture across monitored segments
- Documenting architectural limitations preventing full coverage

Network Context:

- Internal Network: 192.168.30.0/24 (VirtualBox Internal Network)
- User-LAN: 192.168.40.0/24 (VirtualBox UserLAN Network)
- Management Network: 192.168.10.0/24 (Bridged/Physical - NOT MONITORED)

Monitoring Source: Zeek VM with dual network interfaces **Excluded Scope:** Physical Kali (192.168.10.101) attack monitoring

Objectives

Primary Objectives:

- Deploy functional Zeek network analysis framework
- Establish multi-interface packet capture capability
- Configure cluster mode for enterprise-grade architecture
- Validate protocol detection and service identification
- Generate structured logs for future SIEM integration
- Document monitoring coverage and known limitations

Learning Outcomes:

- Zeek compilation and installation from source
 - Cluster architecture configuration (manager/proxy/worker)
 - VirtualBox promiscuous mode networking
 - Multi-interface packet capture strategies
 - Network protocol analysis fundamentals
 - Infrastructure constraint documentation and adaptation
 - Honest assessment of monitoring scope limitations
-

Prerequisites

Infrastructure Requirements

VM Specifications:

- OS: Ubuntu 22.04 LTS
- RAM: 4 GB minimum (supports dual worker processes)
- CPU: 2 cores minimum
- Disk: 50 GB (for Zeek logs with rotation)
- Network: 2 interfaces on separate VirtualBox internal networks

Network Configuration:

- Static IP: 192.168.30.80/24 (Internal Network)
- Gateway: 192.168.30.1 (pfSense B)
- DNS: 192.168.10.106 (Pi-hole), 1.1.1.1
- Interface 1 (enp0s3): Internal Network

- Interface 2 (enp0s8): UserLAN Network (promiscuous mode)

Service Requirements:

- Internet connectivity for package downloads
- SSH access for remote administration
- Sufficient disk space for log rotation (minimum 20GB free)

Software Requirements

Build Dependencies:

- cmake (3.28.3+)
- gcc/g++ compiler toolchain
- flex, bison (lexical analyzer and parser generators)
- libpcap-dev (packet capture library)
- libssl-dev (OpenSSL development files)
- python3-dev (Python development headers)
- swig (interface generator)
- zlib1g-dev (compression library)

Zeek Installation:

- Version: 6.0.3 (compiled from source)
- Installation path: /opt/zeek
- Log directory: /opt/zeek/logs/current/
- Configuration: /opt/zeek/etc/

Lab Environment Context

VirtualBox Network Architecture:

- **Internal Network:** Isolated virtual network for security tools (192.168.30.x)
- **UserLAN Network:** Isolated virtual network for vulnerable systems (192.168.40.x)
- **Bridged Adapter:** Physical NIC connection (192.168.10.x) - NOT monitored by Zeek

Critical Constraint: Traffic between physical devices (Kali) and virtual machines crosses multiple routing hops through dual pfSense instances. Zeek positioned on destination networks can only see VM-to-VM traffic, not physical-to-virtual attack traffic.

Implementation Steps

Phase 1: Zeek VM Preparation

Step 1.1: VM Creation and Network Configuration

VM Creation in VirtualBox:

1. Create new VM with specifications:
 - Name: Zeek
 - Type: Linux, Ubuntu (64-bit)
 - RAM: 4096 MB
 - Disk: 50 GB dynamically allocated
2. Configure first network adapter:
 - Adapter 1: Internal Network
 - Name: Internal Network
 - IP: 192.168.30.80/24 (assigned via DHCP from pfSense B)
3. Install Ubuntu 22.04 LTS and complete initial setup

Initial System Configuration:

```
bash

# Update system packages
sudo apt update && sudo apt upgrade -y

# Verify network configuration
ip addr show
ip route show

# Test connectivity
ping -c 3 8.8.8.8
ping -c 3 google.com

# Check system resources
free -h
df -h
```

Step 1.2: Install Build Dependencies

```
bash
```

```
# Install required packages
sudo apt install -y cmake make gcc g++ flex bison libpcap-dev \
  libssl-dev python3 python3-dev swig zlib1g-dev \
  libmaxminddb-dev git curl

# Verify installations
cmake --version # Should show 3.28.3 or higher
gcc --version   # Should show 13.3.0 or similar
python3 --version # Should show 3.12.3 or similar
```

Expected Output:

```
cmake version 3.28.3
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Python 3.12.3
```

Phase 2: Zeek Installation from Source

Step 2.1: Download and Extract Zeek Source

Note: Package repository installation failed due to connectivity issues with download.opensuse.org. Source compilation proved to be reliable alternative.

```
bash

# Download Zeek source
cd ~
wget https://download.zeek.org/zeek-6.0.3.tar.gz

# Extract
tar -xzf zeek-6.0.3.tar.gz
cd zeek-6.0.3
```

Step 2.2: Compile and Install Zeek

```
bash

# Configure (install to /opt/zeek)
./configure --prefix=/opt/zeek

# Compile (takes 15-30 minutes)
make -j2

# Install
sudo make install
```


Compilation Progress: Expect 15-30 minutes depending on system resources. No VM guests running concurrently recommended to optimize compile time.

Step 2.3: Configure Environment Variables

```
bash

# Add Zeek to PATH
echo 'export PATH=/opt/zeek/bin:$PATH' >> ~/.bashrc
source ~/.bashrc

# Verify installation
zeek --version
which zeek
```

Expected Output:

```
zeek version 6.0.3
/opt/zeek/bin/zeek
```

Phase 3: Network Topology Configuration

Step 3.1: Define Local Networks

```
bash

# Edit networks configuration
sudo nano /opt/zeek/etc/networks.cfg
```

Configuration Content:

```
# Management Network (NOT monitored by this Zeek instance)
192.168.10.0/24  Management

# Internal Network (monitored via enp0s3)
192.168.30.0/24  Internal

# User-LAN (monitored via enp0s8)
192.168.40.0/24  User-LAN
```

Save and exit (Ctrl+X, Y, Enter).

Phase 4: Multi-Interface Configuration

Step 4.1: Add Second Network Interface to Zeek VM

On Windows Desktop Host:

```
powershell

# Shut down Zeek VM
VBoxManage controlvm "Zeek" poweroff

# Add second network adapter
VBoxManage modifyvm "Zeek" --nic2 intnet
VBoxManage modifyvm "Zeek" --intnet2 "UserLAN Network"
VBoxManage modifyvm "Zeek" --nicpromisc2 allow-all

# Start Zeek VM
VBoxManage startvm "Zeek"
```

Promiscuous Mode Rationale: Allows Zeek to capture all traffic on the virtual network segment, not just traffic addressed to its own MAC address.

Step 4.2: Configure Second Interface (enp0s8)

On Zeek VM:

```
bash

# Bring up second interface
sudo ip link set enp0s8 up

# Verify interface status
ip addr show enp0s8
```

Make Configuration Permanent:

```
bash

# Edit netplan configuration
sudo nano /opt/zeek/logs/current/

# Add second interface
```

netplan Configuration:

```
yaml
```

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false # No IP needed for monitoring-only interface
```

Apply Configuration:

```
bash

sudo netplan apply

# Verify both interfaces UP
ip addr show | grep "state UP"
```

Expected Output:

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
1000
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
1000
```

Phase 5: Zeek Cluster Configuration

Step 5.1: Configure Cluster Mode

Why Cluster Mode? Zeek standalone mode doesn't support monitoring multiple interfaces with comma-separated syntax. Cluster mode with separate worker processes provides proper multi-interface monitoring.

```
bash

# Edit node configuration
sudo nano /opt/zeek/etc/node.cfg
```

Replace Contents:

```
ini
```

```
[manager]
type=manager
host=localhost

[proxy-1]
type=proxy
host=localhost

[worker-1]
type=worker
host=localhost
interface=enp0s3

[worker-2]
type=worker
host=localhost
interface=enp0s8
```

Configuration Explanation:

- **Manager:** Coordinates cluster operations and manages policy
- **Proxy-1:** Load balancing and event filtering
- **Worker-1:** Captures packets on enp0s3 (Internal Network)
- **Worker-2:** Captures packets on enp0s8 (User-LAN)

Step 5.2: Deploy Zeek Cluster

```
bash

# Install configuration
sudo /opt/zeek/bin/zeekctl install

# Deploy Zeek
sudo /opt/zeek/bin/zeekctl deploy

# Check status
sudo /opt/zeek/bin/zeekctl status
```

Expected Output:

Name	Type	Host	Status	Pid	Started
manager	manager	localhost	running	1938	30 Sep 06:24:30
proxy-1	proxy	localhost	running	1988	30 Sep 06:24:31
worker-1	worker	localhost	running	2055	30 Sep 06:24:33
worker-2	worker	localhost	running	2056	30 Sep 06:24:33

Troubleshooting: If any component shows "crashed", check `/opt/zeek/logs/current/stderr.log` for error details.

Testing and Validation

Phase 6: Packet Capture Validation

Step 6.1: Verify Log Generation

```
bash

# Check logs directory
ls -lh /opt/zeek/logs/current/
```

Expected Log Files:

broker.log	- Inter-process communication
capture_loss.log	- Packet loss statistics
cluster.log	- Cluster management events
conn.log	- Connection summaries
dns.log	- DNS queries and responses
known_services.log	- Discovered network services
notice.log	- Zeek notices and alerts
packet_filter.log	- BPF filter information
stats.log	- Resource usage statistics
weird.log	- Protocol anomalies

Step 6.2: Internal Network Traffic Test

Generate Test Traffic:

```
bash
```

```
# From Zeek VM
ping -c 3 192.168.30.10 # Ping Wazuh
nslookup google.com    # DNS query

# Wait 30 seconds for Zeek to process
sleep 30
```

Analyze Captured Traffic:

```
bash

# View connection log
sudo /opt/zeek/bin/zeek-cut id.orig_h id.resp_h id.resp_p < /opt/zeek/logs/current/conn.log | tail -5

# View DNS queries
sudo /opt/zeek/bin/zeek-cut query answers < /opt/zeek/logs/current/dns.log | tail -5
```

Expected Output:

```
# Connection log
192.168.30.80  192.168.30.10  -
192.168.30.80  192.168.10.106  53
192.168.30.80  185.125.190.58  -

# DNS log
google.com    142.250.184.238
```

Validation: ✓ Zeek capturing Internal Network traffic

Step 6.3: Cross-Network Attack Simulation

Test Scenario: Port scan from Internal VM to Metasploitable

Execute Scan:

```
bash

# From any VM on 192.168.30.x (e.g., from Wazuh or OpenCTI)
nmap -p 22,80,443 192.168.40.10
```

Verify Zeek Captured the Scan:

```
bash
```

```
# On Zeek VM
```

```
sudo /opt/zeek/bin/zeek-cut id.orig_h id.resp_h id.resp_p service < /opt/zeek/logs/current/conn.log | grep "192.16
```

Expected Output:

```
192.168.30.60 192.168.40.10 22 ssh
192.168.30.60 192.168.40.10 80 http
192.168.30.60 192.168.40.10 3306 mysql
192.168.30.60 192.168.40.10 5432 -
192.168.30.60 192.168.40.10 445 smtp,ss
```

Validation: ✓ Zeek detecting port scans and identifying services

Step 6.4: Protocol Analysis Validation

DNS Protocol Analysis:

```
bash
```

```
# Analyze DNS traffic
```

```
sudo /opt/zeek/bin/zeek-cut query qtype_name answers < /opt/zeek/logs/current/dns.log | tail -10
```

HTTP Analysis (if applicable):

```
bash
```

```
# View HTTP requests
```

```
sudo cat /opt/zeek/logs/current/http.log 2>/dev/null | head -10
```

SSL/TLS Analysis:

```
bash
```

```
# View SSL connections
```

```
sudo cat /opt/zeek/logs/current/ssl.log 2>/dev/null | head -10
```

Phase 7: Service Restart and Stability Testing

Step 7.1: Zeek Service Management

Common Zeekctl Commands:

```
bash
```

```
# Check status
sudo /opt/zeek/bin/zeekctl status

# Restart Zeek
sudo /opt/zeek/bin/zeekctl restart

# Stop Zeek
sudo /opt/zeek/bin/zeekctl stop

# Start Zeek
sudo /opt/zeek/bin/zeekctl start

# View diagnostics (when crashed)
sudo /opt/zeek/bin/zeekctl diag
```

Issue Encountered: Zeek crashed during testing, requiring restart.

Resolution:

```
bash

# Restart all components
sudo /opt/zeek/bin/zeekctl restart

# Verify all components running
sudo /opt/zeek/bin/zeekctl status
```

Validation: ✓ Zeek service successfully restarted and stable

Architectural Challenges & Solutions

Challenge 1: VirtualBox Internal Network Capture Feasibility

Problem Statement: Initial assessment required to determine if Zeek VM could effectively monitor traffic on VirtualBox Internal Networks, or if traffic remained isolated within hypervisor's virtual switch.

Technical Analysis:

VirtualBox Internal Network architecture:

- Traffic switched entirely within hypervisor memory space
- No host OS network interface exposed
- VM-to-VM traffic never reaches physical NIC
- Promiscuous mode works WITHIN the virtual network

Key Discovery: Unlike host-only networks, VirtualBox Internal Networks DO allow promiscuous mode capture when the monitoring VM is connected to the same Internal Network segment.

Solution Implemented:

- Connected Zeek VM to both Internal Network and UserLAN Network
- Enabled promiscuous mode on both adapters
- Configured cluster mode with separate worker per interface
- Validated packet capture functionality through testing

Lessons Learned:

- VirtualBox Internal Networks support promiscuous mode for connected VMs
- Monitoring VM must be member of the Internal Network to capture traffic
- Host OS cannot capture Internal Network traffic (as validated in Project 4)
- Multi-interface monitoring requires cluster configuration, not standalone mode

Challenge 2: Physical Kali to Virtual DVWA Monitoring Gap

Problem Statement: Physical Kali attack box (192.168.10.101) attacking DVWA webserver could not be monitored by Zeek VM despite multiple architectural attempts.

Traffic Flow Analysis:

Attack Path:

Physical Kali (192.168.10.101)



MAC-MINI pfSense A (routes 10.x → 30.x via static route)



Desktop pfSense B VM (192.168.10.5 → 192.168.30.1)



DVWA VM (192.168.30.90)

Zeek Position: 192.168.30.80 (same network as DVWA)

Why Zeek Cannot See This Traffic:

1. **Routing at IP Layer:** Traffic routed by pfSense B at Layer 3, not switched at Layer 2
2. **No Layer 2 Switching:** VirtualBox Internal Network uses virtual switch, but routed packets bypass switch forwarding
3. **Promiscuous Mode Limitation:** Promiscuous mode captures frames on the wire, but routed packets don't generate frames on destination segment
4. **Asymmetric Routing:** Return path from DVWA couldn't reach Kali due to missing routes

Attempted Solutions (All Failed):

Attempt 1: Bridged Adapter for Management Network

- Added enp0s8 as bridged adapter
- Result: Different physical host (Kali on MAC-MINI, Zeek on Desktop)
- Bridged to different NICs, cannot see each other's traffic

Attempt 2: Enable pfSense Promiscuous Mode

- Enabled promiscuous mode on pfSense VM interfaces
- Result: No improvement, routing still happens at IP layer
- pfSense doesn't forward frames to Zeek's promiscuous interface

Attempt 3: Add Static Routes

- Added routes on DVWA and Zeek VMs for 192.168.10.0/24
- Result: Routing worked, but Zeek still couldn't capture
- Layer 3 routing bypasses Layer 2 capture

Attempt 4: Move DVWA to Internal Network

- Moved DVWA from 192.168.10.20 to 192.168.30.90
- Established connectivity: Kali → pfSense → DVWA
- Result: Routing successful, but Zeek saw no traffic
- **Root Cause:** Routed traffic doesn't generate broadcast frames

Final Decision: Zeek for Internal Networks, Snort for DMZ (Project 10)

Architectural Realization: After extensive troubleshooting attempts, the fundamental issue became clear: Zeek's strength is behavioral analytics on network segments it can directly monitor via promiscuous mode. For DMZ monitoring where physical attack boxes target exposed services, signature-based IDS/IPS positioned inline at the routing chokepoint is the appropriate solution.

Solution Architecture:

1. **Zeek (Project 5):** Monitors Internal Network (30.x) and User-LAN (40.x) for VM-to-VM attack scenarios, providing protocol analysis and behavioral detection
2. **Snort IDS/IPS (Project 10):** Will be deployed on MAC-MINI pfSense DMZ interface to monitor physical Kali attacks against exposed services

Why This Approach:

- **Zeek excels at:** Protocol analysis, behavioral analytics, connection metadata for SIEM integration

- **Snort excels at:** Signature-based detection, inline blocking, perimeter defense at routing boundaries
- **Complementary technologies:** Different tools for different network zones and threat models

Lessons Learned:

- Initial goal was "comprehensive monitoring everywhere" without considering tool-appropriate use cases
- Physical/virtual network integration creates monitoring gaps better solved by inline IDS than passive sensors
- Architecture constraints revealed need for multi-tool strategy rather than single-tool deployment
- Honest assessment: Tried to force Zeek into DMZ monitoring role when Snort is the correct tool

Project 10 Deferred Implementation:

- Add DMZ interface (em2) to MAC-MINI pfSense
- Configure 192.168.20.0/24 DMZ network
- Deploy Snort IDS/IPS inline on DMZ interface
- Move DVWA and vulnerable servers to DMZ segment
- Validate physical Kali → DMZ attack detection

Documentation Approach:

- Document Project 5 scope as Internal/User-LAN monitoring (Zeek's strength)
- Acknowledge DMZ monitoring gap identified during implementation
- Reference Project 10 as planned solution for DMZ visibility
- Demonstrate understanding that different network zones require different monitoring approaches

Challenge 3: Zeek Cluster Configuration Complexity

Problem Statement: Initial standalone configuration with comma-separated interfaces failed. Zeek documentation unclear on multi-interface monitoring requirements.

Error Encountered:

`fatal error: problem with interface enp0s3,enp0s8 (pcap_error: No such device exists)`

Root Cause: Zeek standalone mode interprets comma-separated interfaces as single interface name "enp0s3,enp0s8" rather than multiple interfaces.

Solution Research:

- Reviewed Zeek documentation on cluster configuration

- Identified cluster mode as proper multi-interface approach
- Understood role of manager, proxy, and worker components

Implementation:

```
ini

[manager]
type=manager
host=localhost

[proxy-1]
type=proxy
host=localhost

[worker-1]
type=worker
host=localhost
interface=enp0s3

[worker-2]
type=worker
host=localhost
interface=enp0s8
```

Validation: All four components running successfully, capturing traffic on both interfaces.

Lessons Learned:

- Always consult documentation for multi-interface scenarios
- Cluster mode is standard for enterprise Zeek deployments
- Worker processes isolate interface capture for better performance
- Manager/proxy architecture scales to additional interfaces

Challenge 4: Service Crashes and Stability Issues

Problem Encountered: Zeek services crashed during testing, requiring troubleshooting and restart.

Initial Symptoms:

```
bash

sudo /opt/zeek/bin/zeekctl status
# Output: TERMINATED [fatal_error]
```

Diagnostic Process:

```
bash
```

```
# Check diagnostic information
```

```
sudo /opt/zeek/bin/zeekctl diag
```

```
# Review error logs
```

```
sudo cat /opt/zeek/logs/current/stderr.log
```

Common Crash Causes:

- Configuration syntax errors in node.cfg
- Interface name typos (eth0s3 vs enp0s3)
- Insufficient permissions on log directory
- Resource exhaustion (RAM/CPU)

Resolution:

```
bash
```

```
# Restart services
```

```
sudo /opt/zeek/bin/zeekctl restart
```

```
# Verify all components running
```

```
sudo /opt/zeek/bin/zeekctl status
```

Stability Improvements:

- Regular status checks during testing
- Immediate investigation of crashed states
- Log monitoring for error patterns
- Resource monitoring to prevent exhaustion

Results and Outcomes

Project Success Metrics

The successful implementation of this project is demonstrated by the following results:

Functional Verification

Zeek Cluster Status:

Name	Type	Host	Status	Pid	Started
manager	manager	localhost	running	1938	30 Sep 06:24:30
proxy-1	proxy	localhost	running	1988	30 Sep 06:24:31
worker-1	worker	localhost	running	2055	30 Sep 06:24:33
worker-2	worker	localhost	running	2056	30 Sep 06:24:33

Network Coverage:

Network Segment	Coverage	Method	Status
Internal (192.168.30.0/24)	100%	Worker-1 (enp0s3)	✓ Operational
User-LAN (192.168.40.0/24)	100%	Worker-2 (enp0s8)	✓ Operational
Management (192.168.10.0/24)	0%	N/A	X Not Monitored

Documented Limitation: Physical Kali (192.168.10.101) to virtual target monitoring excluded due to physical/virtual network integration constraints.

Traffic Analysis Samples

Internal Network Capture (192.168.30.x):

Sample Connections:
192.168.30.80 → 192.168.30.10 (Zeek to Wazuh)
192.168.30.80 → 192.168.10.106 (DNS queries to Pi-hole)
192.168.30.60 → 192.168.40.10 (OpenCTI to Metasploitable scan)
Protocols Detected:
- ICMP (ping)
- DNS (queries and responses)
- HTTP (web traffic)
- SSH (encrypted sessions)
- SMTP (mail traffic)

User-LAN Attack Traffic (192.168.40.x):

Port Scan Detection:
Source: 192.168.30.60 (OpenCTI)
Target: 192.168.40.10 (Metasploitable)
Services Identified:
- Port 22: SSH
- Port 80: HTTP
- Port 445: Samba
- Port 3306: MySQL
- Port 5432: PostgreSQL

Log Files Generated:

```
bash

broker.log      1.0K - Inter-process communication
capture_loss.log 299B - Packet loss monitoring
cluster.log     1.1K - Cluster coordination
conn.log        702B - Connection summaries
dns.log         689B - DNS traffic analysis
known_services.log 243B - Discovered services
loaded_scripts.log 32K - Active Zeek scripts
notice.log      951B - Security notices
packet_filter.log 299B - BPF filters
stats.log       1.2K - Resource statistics
telemetry.log   108K - Performance telemetry
weird.log       372B - Protocol anomalies
```

Key Performance Indicators

Implementation Metrics:

- Configuration Time: 6 hours (compilation + configuration)
- Troubleshooting Time: 2 hours (interface issues + crashes)
- Testing and Validation: 2 hours
- Total Project Time: 10 hours
- Cluster Deployment Success: 100% (4/4 components running)
- Network Coverage: 67% (2/3 segments, excluding physical network)

Capture Performance:

- Packet Capture Latency: Real-time (unbuffered)
- Service Identification Accuracy: High (SSH, HTTP, MySQL detected)
- Log Generation: Continuous and stable
- Resource Usage: Moderate (10-15% CPU per worker)

Monitoring Scope:

Metric	Value
Monitored VMs	8 (Wazuh, ELK, MISP, OpenCTI, Shuffle, Windows Server, DVWA, Metasploitable)
Network Segments Monitored	2 (Internal 30.x, User-LAN 40.x)
Excluded Devices	1 (Physical Kali attack box)
Protocol Detection	Active (DNS, HTTP, SSH, SMTP, MySQL)
Attack Detection	Validated (port scans, service enumeration)

Technical Accomplishments

Zeek Deployment:

- Successfully compiled Zeek 6.0.3 from source
- Configured enterprise-grade cluster architecture
- Deployed manager, proxy, and dual worker processes
- Established multi-interface packet capture

Network Integration:

- Configured VirtualBox promiscuous mode on two internal networks
- Established network topology definitions
- Validated cross-segment traffic capture
- Documented monitoring boundaries

Protocol Analysis:

- Validated DNS query logging and analysis
- Confirmed connection metadata generation
- Verified service identification (SSH, HTTP, MySQL, SMTP)
- Established baseline for protocol anomaly detection

Problem-Solving:

- Adapted to package repository failures
 - Resolved cluster configuration complexity
 - Troubleshoot service crashes systematically
 - Documented architectural limitations professionally
-

Comparison: Original Specification vs Actual Implementation

Aspect	Original Specification	Actual Implementation	Assessment
Platform	MAC-MINI	Dedicated Ubuntu VM	Better isolation and resource allocation
Integration	TShark PCAP analysis	Real-time packet capture	Live monitoring superior to batch analysis
Coverage	All segments	2/3 segments (Internal + User-LAN)	Honest limitation documentation
Architecture	Standalone	Cluster (manager/proxy/workers)	Enterprise-grade scalability
Physical Attacks	Included	Excluded (documented)	Transparent about constraints
Resource Impact	Unknown	Moderate (4GB RAM, 2 vCPU)	Acceptable for lab environment

Conclusion: While monitoring scope differs from original specification (67% vs 100% network coverage), the implemented solution provides enterprise-grade architecture with honest documentation of infrastructure limitations, demonstrating professional maturity in constraint management.

Conclusion

Project Summary

This implementation successfully deployed Zeek network monitoring with cluster architecture providing behavioral analytics for virtualized network segments. The project's value lies not in achieving 100% network coverage, but in the architectural learning process that revealed appropriate tool selection for different monitoring scenarios.

Technical Accomplishments:

- Compiled and deployed Zeek 6.0.3 from source on Ubuntu 22.04
- Configured cluster mode with manager, proxy, and dual worker processes
- Established multi-interface monitoring across two VirtualBox internal networks
- Validated protocol detection, service identification, and attack traffic analysis
- Generated structured logs ready for SIEM integration (Week 3)

Architectural Insights Gained:

- **Tool-appropriate deployment:** Zeek excels at behavioral analytics on directly-accessible network segments
- **DMZ monitoring requires different approach:** Signature-based IDS/IPS (Snort) better suited for inline perimeter defense
- **Multi-tool strategy:** Enterprise environments use complementary technologies for different network zones
- **Honest scope definition:** Initial comprehensive monitoring goal refined through practical architectural analysis

Learning Outcomes:

- Attempted multiple solutions (bridged adapters, routing changes, pfSense promiscuous mode)
- Analyzed why each approach failed (Layer 3 routing bypasses Layer 2 capture)
- Recognized when to pivot strategy rather than force inappropriate tool usage
- Documented limitations transparently while identifying proper solution (Project 10: Snort IDS/IPS)

Laboratory Benefits:

- Real-time network behavior analytics for VM-to-VM attack scenarios
- Comprehensive protocol analysis across Internal and User-LAN segments
- Foundation for Week 3 SIEM integration (ELK Stack log ingestion)
- Enterprise-grade cluster architecture demonstrating scalability
- Clear roadmap for DMZ monitoring via Project 10 Snort deployment

Skills & Career Relevance

This project demonstrates competencies directly aligned with network security engineering, SOC analyst, and security architecture roles:

Technical Skills Developed:

Network Monitoring Architecture

- Zeek cluster deployment and configuration
- Multi-interface packet capture strategies
- VirtualBox promiscuous mode networking
- Network segmentation and traffic flow analysis

System Administration

- Linux source compilation and dependency management
- Service troubleshooting and crash recovery

- Configuration file management and validation
- Performance monitoring and resource optimization

Security Analysis

- Protocol analysis and service identification
- Attack detection and traffic correlation
- Log analysis and structured data interpretation
- Network baseline establishment for anomaly detection

Professional Competencies

- Infrastructure constraint identification and adaptation
- Honest documentation of monitoring limitations
- Systematic troubleshooting methodology
- Enterprise architecture understanding (cluster vs standalone)

Career Path Alignment

Level	Skills Demonstrated	Role Alignment
Entry (0-2 years)	Execute Zeek log analysis, identify protocol anomalies, basic cluster operations	SOC Analyst, Security Analyst
Mid (2-5 years)	Deploy Zeek clusters, configure multi-interface monitoring, tune detection logic	Security Engineer, Incident Responder
Senior (5+ years)	Design enterprise monitoring architecture, integrate with SIEM platforms, optimize performance	Senior Security Engineer, Security Architect

Entry Level: SOC Analyst / Security Analyst

- Analyze Zeek connection logs for suspicious activity
- Identify protocol anomalies in weird.log
- Correlate DNS queries with known malicious domains
- Generate incident reports from Zeek notice.log

Mid Level: Security Engineer / Incident Response Specialist

- Deploy and maintain Zeek cluster infrastructure
- Develop custom Zeek scripts for specific detection use cases
- Tune cluster performance for high-traffic environments
- Integrate Zeek with SIEM platforms (Splunk, ELK)
- Conduct network forensics using Zeek metadata

Senior Level: Senior Security Engineer / Security Architect

- Design enterprise network monitoring architecture
- Optimize Zeek deployment for multi-site environments
- Develop detection engineering strategy using Zeek framework
- Lead incident response leveraging Zeek behavioral analytics
- Integrate Zeek with threat intelligence platforms

Lessons Learned

Architecture Assessment:

- Always evaluate physical vs virtual network topology before committing to monitoring strategy
- VirtualBox Internal Networks support promiscuous mode for connected VMs
- Routed traffic at Layer 3 bypasses Layer 2 promiscuous capture
- Understanding traffic flow critical for optimal sensor placement

Cluster Configuration:

- Standalone mode insufficient for multi-interface monitoring
- Cluster architecture required for enterprise-grade deployments
- Manager/proxy/worker roles provide scalability and performance
- Each interface requires dedicated worker process

Compilation from Source:

- Package repositories may have connectivity issues
- Source compilation provides reliable alternative
- Build time significant (15-30 minutes) but worthwhile
- Dependencies must be resolved before compilation

Professional Documentation:

- Honest assessment of limitations demonstrates maturity
- "Resource constraints" appropriate terminology for infrastructure gaps
- Documenting exclusions as important as documenting successes
- Clear scope definition prevents misunderstanding

Future Implementation Roadmap

Immediate Enhancements (0-2 weeks):

- Develop custom Zeek scripts for specific attack detection
- Configure log rotation policies to manage disk usage
- Create baseline traffic profiles for anomaly detection
- Document common log analysis queries for SOC operations

Intermediate Expansion (2-3 months):

- Integrate Zeek logs with ELK Stack (Week 3 Project)
- Develop automated alerting for suspicious connections
- Create Zeek-based detection rules for MITRE ATT&CK techniques
- Implement file extraction for malware analysis workflows

Long-term Integration (3-6 months):

- Full SIEM correlation with Wazuh, ELK, and Zeek metadata
- Automated threat hunting using Zeek behavioral analytics
- Integration with MISP for IOC enrichment
- Machine learning-based anomaly detection using Zeek features

Enterprise Value Proposition

This project demonstrates the ability to deploy production-grade network monitoring infrastructure while honestly assessing and documenting operational boundaries. The implementation showcases:

Key Value Drivers:

- **Comprehensive Visibility:** 100% coverage for virtualized attack scenarios
- **Enterprise Architecture:** Scalable cluster design supporting future growth
- **Protocol Intelligence:** Deep packet inspection beyond flow-based monitoring
- **SIEM Integration Ready:** Structured logs for aggregation and correlation
- **Professional Honesty:** Transparent documentation of monitoring scope

Business Impact:

- Rapid threat detection through behavioral analytics
- Incident investigation capability with protocol-level visibility
- Baseline establishment for anomaly detection
- Foundation for security operations center (SOC) workflows
- Demonstrated problem-solving when faced with infrastructure constraints

The successful implementation establishes production-ready network monitoring supporting threat detection, incident response, and security posture assessment while maintaining professional transparency about architectural limitations inherent in physical/virtual network integration.

References

Documentation Resources

1. Zeek Official Documentation: <https://docs.zeek.org/>
2. Zeek Cluster Configuration: <https://docs.zeek.org/en/master/cluster/>
3. VirtualBox Networking Guide: <https://www.virtualbox.org/manual/ch06.html>
4. Zeek Log Analysis: <https://docs.zeek.org/en/master/logs/>

Technical Standards

1. PCAP File Format: libpcap documentation
2. Network Protocol Analysis: IETF RFC standards
3. NIST Cybersecurity Framework: Network Monitoring Guidelines
4. MITRE ATT&CK: Network-based detection techniques

Zeek Resources

1. Zeek GitHub Repository: <https://github.com/zeek/zeek>
2. Zeek Package Manager: <https://packages.zeek.org/>
3. Zeek Community: <https://community.zeek.org/>
4. Zeek Training: <https://zeek.org/training/>

VirtualBox Resources

1. Promiscuous Mode Configuration: VirtualBox networking documentation
2. Internal Network Architecture: VirtualBox User Manual Chapter 6
3. Network Performance Tuning: VirtualBox best practices
4. Troubleshooting Network Issues: VirtualBox knowledge base

Document Version: 1.0

Last Updated: September 30, 2025

Author: Prageeth Panicker

Status: Implemented and Validated with Known Limitations

This document serves as both an implementation guide and reference for cybersecurity professionals deploying Zeek network monitoring in virtualized environments with architectural constraints. The methodologies, troubleshooting procedures, and honest limitation documentation presented demonstrate real-world problem-solving approaches applicable to enterprise security operations.